Homework 2: Exploring patterns of environmental justice

Courtney Lorey

2026-10-21

Part 1

Objective

Using 2023 data from the United States Environmental Protection Agency's former Environmental Justice Screening and Mapping Tool (EJScreen) as well as data from the Mapping Inequality project at the University of Richmond's Digital Scholarship Lab, this section explores historical redlining in Los Angeles and its legacy on present-day environmental justice.

Load packages

```
library(here)
library(tidyverse)
library(sf)
library(tmap)
library(gt)
```

Import data

```
# Read in geodatabase of EJScreen data at the Census Block Group level
ejscreen <- sf::st_read("/Users/courtneylorey/Desktop/GitHub/eds223/eds223-hw2/
data/ejscreen/EJSCREEN_2023_BG_StatePct_with_AS_CNMI_GU_VI.gdb")
```

```
Reading layer `EJSCREEN_StatePctiles_with_AS_CNMI_GU_VI' from data source
         `/Users/courtneylorey/Desktop/GitHub/eds223/eds223-hw2/data/ejscreen/
EJSCREEN_2023_BG_StatePct_with_AS_CNMI_GU_VI.gdb'
  using driver `OpenFileGDB'
Simple feature collection with 243021 features and 223 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -19951910 ymin: -1617130 xmax: 16259830 ymax: 11554350
Projected CRS: WGS 84 / Pseudo-Mercator
```

```
# Filter to LA County
los_angeles <- ejscreen |>
  dplyr::filter(ST_ABBREV == "CA") |>
  dplyr::filter(CNTY_NAME %in% c("Los Angeles County"))

# Read in json file
redline <- sf::st_read("/Users/courtneylorey/Desktop/GitHub/eds223/eds223-hw2/
data/mapping-inequality/mapping-inequality-los-angeles.json")
```

```
Reading layer `mapping-inequality-los-angeles' from data source
          `/Users/courtneylorey/Desktop/GitHub/eds223/eds223-hw2/data/mapping-
inequality/mapping-inequality-los-angeles.json'
  using driver `GeoJSON'
Simple feature collection with 417 features and 14 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -118.6104 ymin: 33.70563 xmax: -117.7028 ymax: 34.30388
Geodetic CRS:  WGS 84
```

```
# Check that the CRS matches for ejscreen and redline
if(st_crs(redline) == st_crs(ejscreen)) {
  print("CRS matches")
} else {
  warning('CRS does not match')
  print(paste('redline CRS is', st_crs(redline)))
  print(paste('ejscreen CRS is', st_crs(ejscreen)))
}
```

```
Warning: CRS does not match
```

```
[1] "redline CRS is WGS 84"
[2]  "redline  CRS  is  GEOGCRS[\"WGS  84\",\n        DATUM[\"World  Geodetic
System  1984\",\n                ELLIPSOID[\"WGS  84\",6378137,298.257223563,
\n                  LENGTHUNIT[\"metre\",1]]],\n      PRIMEM[\"Greenwich\",0,
\n         ANGLEUNIT[\"degree\",0.0174532925199433]],\n     CS[ellipsoidal,2],
\n          AXIS[\"geodetic latitude (Lat)\",north,\n               ORDER[1],
\n                                ANGLEUNIT[\"degree\",0.0174532925199433]],\n
AXIS[\"geodetic  longitude  (Lon)\",east,\n                         ORDER[2],\n
ANGLEUNIT[\"degree\",0.0174532925199433]],\n    ID[\"EPSG\",4326]]"
[1] "ejscreen CRS is WGS 84 / Pseudo-Mercator"
[2] "ejscreen CRS is PROJCRS[\"WGS 84 / Pseudo-Mercator\",\n   BASEGEOGCRS[\"WGS
```

```
84\",\n                ENSEMBLE[\"World Geodetic System 1984 ensemble\",\n
MEMBER[\"World Geodetic System 1984 (Transit)\"],\n            MEMBER[\"World
Geodetic System 1984 (G730)\"],\n                  MEMBER[\"World Geodetic
System 1984 (G873)\"],\n                MEMBER[\"World Geodetic System 1984
(G1150)\"],\n              MEMBER[\"World Geodetic System 1984 (G1674)\"],\n
MEMBER[\"World Geodetic System 1984 (G1762)\"],\n              MEMBER[\"World
Geodetic System 1984 (G2139)\"],\n             MEMBER[\"World Geodetic System
1984 (G2296)\"],\n                ELLIPSOID[\"WGS 84\",6378137,298.257223563,
\n              LENGTHUNIT[\"metre\",1]],\n          ENSEMBLEACCURACY[2.0]],\n
PRIMEM[\"Greenwich\",0,\n           ANGLEUNIT[\"degree\",0.0174532925199433]],
\n              ID[\"EPSG\",4326]],\n       CONVERSION[\"Popular Visualisation
Pseudo-Mercator\",\n         METHOD[\"Popular Visualisation Pseudo Mercator\",
\n                  ID[\"EPSG\",1024]],\n            PARAMETER[\"Latitude of
natural origin\",0,\n              ANGLEUNIT[\"degree\",0.0174532925199433],\n
ID[\"EPSG\",8801]],\n          PARAMETER[\"Longitude of natural origin\",0,\n
ANGLEUNIT[\"degree\",0.0174532925199433],\n                ID[\"EPSG\",8802]],
\n        PARAMETER[\"False easting\",0,\n            LENGTHUNIT[\"metre\",1],
\n               ID[\"EPSG\",8806]],\n          PARAMETER[\"False northing\",0,
\n              LENGTHUNIT[\"metre\",1],\n              ID[\"EPSG\",8807]]],\n
CS[Cartesian,2],\n         AXIS[\"easting (X)\",east,\n              ORDER[1],
\n             LENGTHUNIT[\"metre\",1]],\n        AXIS[\"northing (Y)\",north,
\n             ORDER[2],\n             LENGTHUNIT[\"metre\",1]],\n    USAGE[\n
SCOPE[\"Web mapping and visualisation.\"],\n       AREA[\"World between 85.06°S
and 85.06°N.\"],\n       BBOX[-85.06,-180,85.06,180]],\n    ID[\"EPSG\",3857]]"
```

```
# Update redline crs to match ejscreen and run the test again
redline <-st_transform(redline, 3857)

# Check that the CRS matches for ejscreen and redline
if(st_crs(redline) == st_crs(ejscreen)) {
  print("CRS matches")
} else {
  warning('CRS does not match')
  print(paste('redline CRS is', st_crs(redline)))
  print(paste('ejscreen CRS is', st_crs(ejscreen)))
}
```

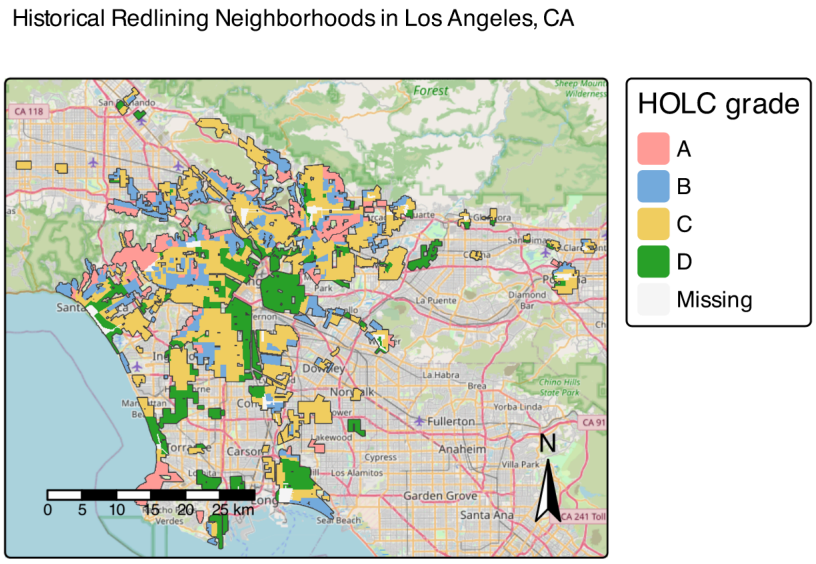
```
[1] "CRS matches"
```

Visualize data

```
  # Create a map of historical redlining neighborhoods, including neighborhoods
colored by HOLC grade, an appropriate base map, a clear title and legend

tm_shape(redline) +
  tm_basemap("OpenStreetMap") +
  tm_polygons() +
  tm_fill(
    fill = "grade",
   fill.legend = tm_legend(title = "HOLC grade", position = tm_pos_out("right"))
  ) +
  tm_title(text = "Historical Redlining Neighborhoods in Los Angeles, CA") +
  tm_compass() +
  tm_scalebar(position = c("left", "bottom")) +
  tm_credits("Map 1", position = tm_pos_out("center", "bottom"));
```

```
[plot mode] fit legend/component: Some legend items or map compoments do not
fit well, and are therefore rescaled.
ℹ Set the tmap option `component.autoscale = FALSE` to disable rescaling.
```



Map 1

```
# Join LA and redline spatial data to create a summary
la_redline <- st_join(los_angeles, redline, join = st_intersects)
```

```r
# Create the summary data frame
la_block_groups <- la_redline |>
  st_drop_geometry() |>
  mutate(Grade = grade) |>
  group_by(Grade) |>
  summarize(`Block Count` = n()) |>
  mutate(Grade = replace_na(Grade, "Missing")) |>
  mutate(`% of Census Blocks` = (`Block Count` / sum(`Block Count`)) * 100)

# Convert the data frame to a table using gt()
la_block_groups |>
  gt()
```

Grade	Block Count	% of Census Blocks
A	449	4.99555
B	1239	13.78505
C	3058	34.02314
D	1346	14.97552
Missing	2896	32.22074

```r
# Summarize average conditions by grade
avg_low_inc <- la_redline |>
  st_drop_geometry() |>
  group_by(grade) |>
  drop_na(grade) |>
  summarise(avg_pct_low_inc = mean(LOWINCPCT, na.rm = TRUE))

avg_pm25 <- la_redline |>
  st_drop_geometry() |>
  group_by(grade) |>
  drop_na(grade) |>
  summarise(avg_pct_pm25 = mean(P_PM25, na.rm = TRUE))

avg_life_exp <-  la_redline |>
  st_drop_geometry() |>
  group_by(grade) |>
  drop_na(grade) |>
  summarise(avg_pct_life_exp = mean(LIFEEXPPCT, na.rm = TRUE))
```

```r
# Visualize average conditions by grade
ggplot(
  data = avg_low_inc,
  mapping = aes(
    x = grade,
    y = avg_pct_low_inc,
    fill = grade
  )
) +
geom_col() +
theme_bw() +
labs(x = "HOLC grade", y = "Mean percent low income", title = "Percent low income by HOLC grade", caption = "Figure 1")
```

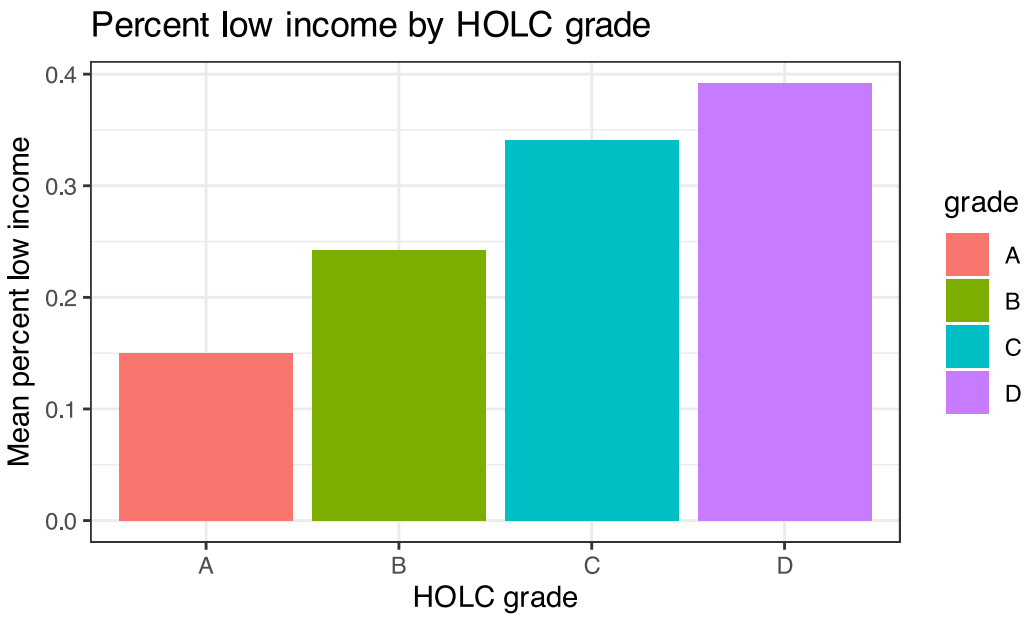


Figure 1

```r
ggplot(
  data = avg_pm25,
  mapping = aes(
    x = grade,
    y = avg_pct_pm25,
    fill = grade
  )
) +
geom_col() +
```

```
theme_bw() +
labs(x = "HOLC grade", y = "Mean percent PM 2.5", title = "Percent PM 2.5 by
HOLC grade", caption = "Figure 2")
```

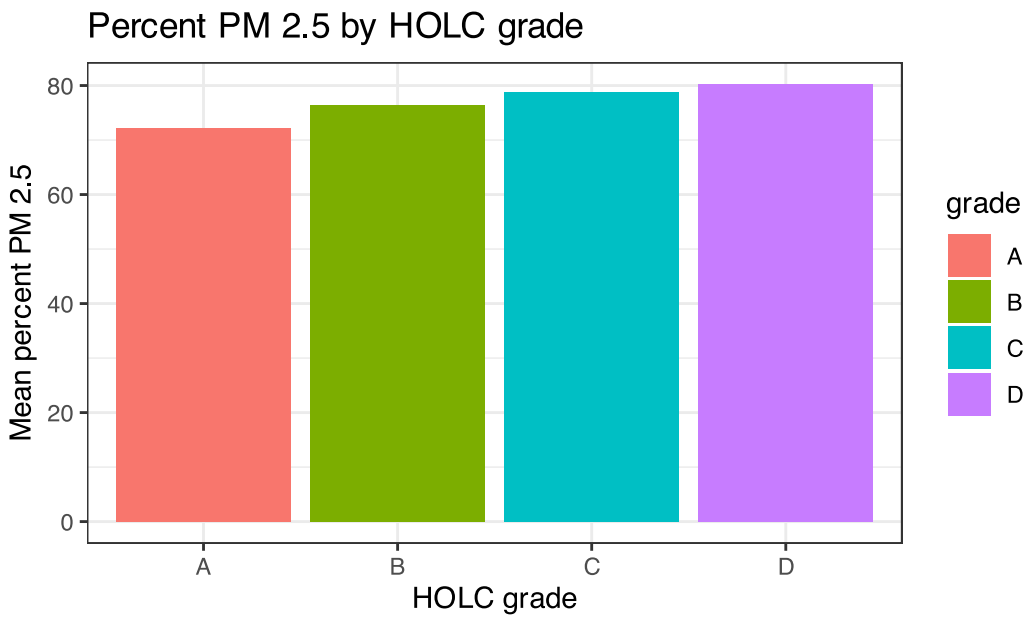


Figure 2

```
ggplot(
  data = avg_life_exp,
  mapping = aes(
    x = grade,
    y = avg_pct_life_exp,
    fill = grade
  )
) +
geom_col() +
theme_bw() +
labs(x = "HOLC grade", y = "Mean percent low life expectancy", title = "Percent
low life expectancy by HOLC grade", caption = "Figure 3")
```



Figure 3

Interpretation

Map 1 utilizes the Los Angeles redlining spatial data to visualize the historical redlining neighborhoods across Los Angeles, California. Figures 1-3 utilize 2023 EJScreen data (specifically average percent low income, average percent PM 2.5 pollution, and average percent low life expectancy) combined with Los Angeles redlining data to look at how these conditions vary by Home Owners' Loan Corporation (HOLC) grade in LA. Across all three figures, the proportion of negative attributes increases from grade A to grade D - meaning that grade D has the highest average percent of low income, the highest average percent of PM 2.5 pollution, and the highest average percent of low life expectancy, while grade A has the lowest proportion of these negative conditions. This is consistent with the finding that historical redlining practices have negative and lasting consequences on community wealth and community health.

Part 2

Objective

Utilizing data from the Mapping Inequality project at the University of Richmond's Digital Scholarship Lab as well as data from the Statewide Integrated Traffic Records System (SWITRS) database, this section explores the legacy of historical redlining in Los Angeles on the occurrence of pedestrian and bike crashes/fatalities from 2020 to 2025.

Import data

```
# Read in crashes data, and inspect lat/long columns
crash_test <- read_csv("/Users/courtneylorey/Desktop/GitHub/eds223/eds223-hw2/
data/SWITRS_LA/Crashes.csv")
```

```
Rows: 251100 Columns: 80
──                                                                    Column
specification ──────────────────────────────────────────────────────────────
Delimiter: ","
chr  (45): COLLISION_TIME, OFFICER_ID, REPORTING_DISTRICT, CHP_BEAT_TYPE, CI...
dbl  (33): CASE_ID, ACCIDENT_YEAR, JURIS, DAY_OF_WEEK, CHP_SHIFT, POPULATION...
date  (2): PROC_DATE, COLLISION_DATE

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
sum(is.na(crash_test$POINT_X))
```

```
[1] 9484
```

```
sum(is.na(crash_test$POINT_Y))
```

```
[1] 9484
```

```
sum(is.na(crash_test$LATITUDE))
```

```
[1] 97350
```

```
sum(is.na(crash_test$LONGITUDE))
```

```
[1] 97350
```

```
# Import the crashes data as sf
crashes  <-  read_csv("/Users/courtneylorey/Desktop/GitHub/eds223/eds223-hw2/
data/SWITRS_LA/Crashes.csv") |>
  as_tibble() |>
  drop_na(POINT_X) |>
```

```
  drop_na(POINT_Y) |>
  rename(long = POINT_X,
         lat = POINT_Y) |> # Use POINT_X and POINT_Y as lat/long (as it has far
fewer NAs)
  st_as_sf(coords = c("long", "lat"), crs = 4326, remove = FALSE) |>
  st_transform(crs = 3857) # Set crs to match that of redline (3857)
```

```
Rows: 251100 Columns: 80
──                                                                  Column
specification ──────────────────────────────────────────────────────────────
Delimiter: ","
chr  (45): COLLISION_TIME, OFFICER_ID, REPORTING_DISTRICT, CHP_BEAT_TYPE, CI...
dbl  (33): CASE_ID, ACCIDENT_YEAR, JURIS, DAY_OF_WEEK, CHP_SHIFT, POPULATION...
date  (2): PROC_DATE, COLLISION_DATE

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Visualize data

```
# Before joining, check that the datasets have matching CRS
if(st_crs(redline) == st_crs(crashes)) {
  print("CRS matches")
} else {
  warning('CRS does not match')
  print(paste('redline CRS is', st_crs(redline)))
  print(paste('crashes CRS is', st_crs(crashes)))
}
```

```
[1] "CRS matches"
```

```
# Join redline data with crash data
redline_crashes <- st_join(crashes, redline)

# Select relevant columns from crash data
filtered_redline_crash <- redline_crashes |>
  st_drop_geometry() |>
  mutate(
    grade = ifelse(is.na(grade), "HOLC Grade Missing", grade)
  ) |>
  select(grade, long, lat, PEDESTRIAN_ACCIDENT, BICYCLE_ACCIDENT) |>
```

```
  filter(PEDESTRIAN_ACCIDENT == "Y" | BICYCLE_ACCIDENT == "Y") |>
  count(grade) |>
  mutate(Percent = (n / sum(n)) * 100)

# Convert the data frame to a table using gt()
gt(filtered_redline_crash)
```

grade	n	Percent
A	360	1.087646
B	1782	5.383848
C	9026	27.269706
D	4532	13.692257
HOLC Grade Missing	17399	52.566543

```
# Visualize
ggplot(
  data = filtered_redline_crash,
  mapping = aes(
    x = grade,
    y = Percent,
    fill = grade
  )
) +
geom_col() +
theme_bw() +
labs(x = "HOLC grade", y = "Percent accidents", title = "Percentage of pedestrian and bike crashes and fatalities by HOLC grade", fill = "Grade", caption = "Figure 4")
```

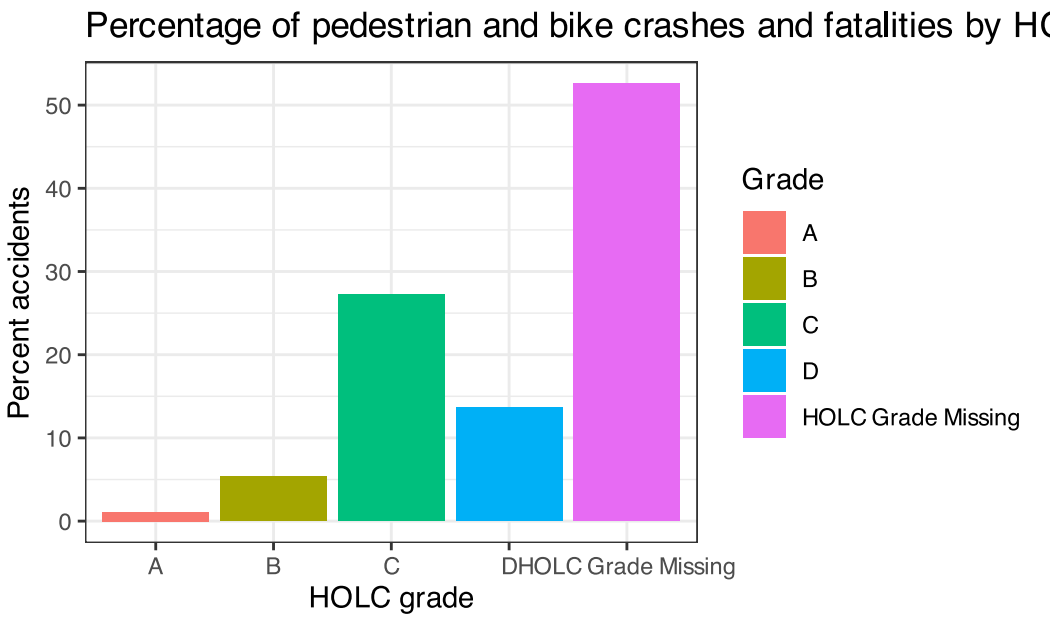


Figure 4

Interpretation

Figure 4 shows that for pedestrian and bike accidents (including crashes and fatalities) that occurred in Los Angeles within a given HOLC-graded area between 2020 and 2025, the largest proportion of accidents occured within HOLC grades C (27%) and D (14%). HOLC grades C and D experienced significantly more accidents than HOLC grades A and B. This demonstrates an outsized occurence of accidents in areas with lower HOLC grades, indicating that these redlined neighborhoods are at a higher risk of accidents, and that highly graded areas are safer from accidents. This is consistent with the finding that historical redlining continues to have detrimental impacts on present-day community health.

It is also worth noting that the majority (52%) of accidents occurred in areas that did not have an assigned HOLC grade (grade was listed as NA in the data set).